

Phân tích & Thiết kế thuật toán (Algorithms Design & Analysis)

L/O/G/O

GV: HUỖNH THỊ THANH THƯỜNG

Email: thuonghtt@uit.edu.vn

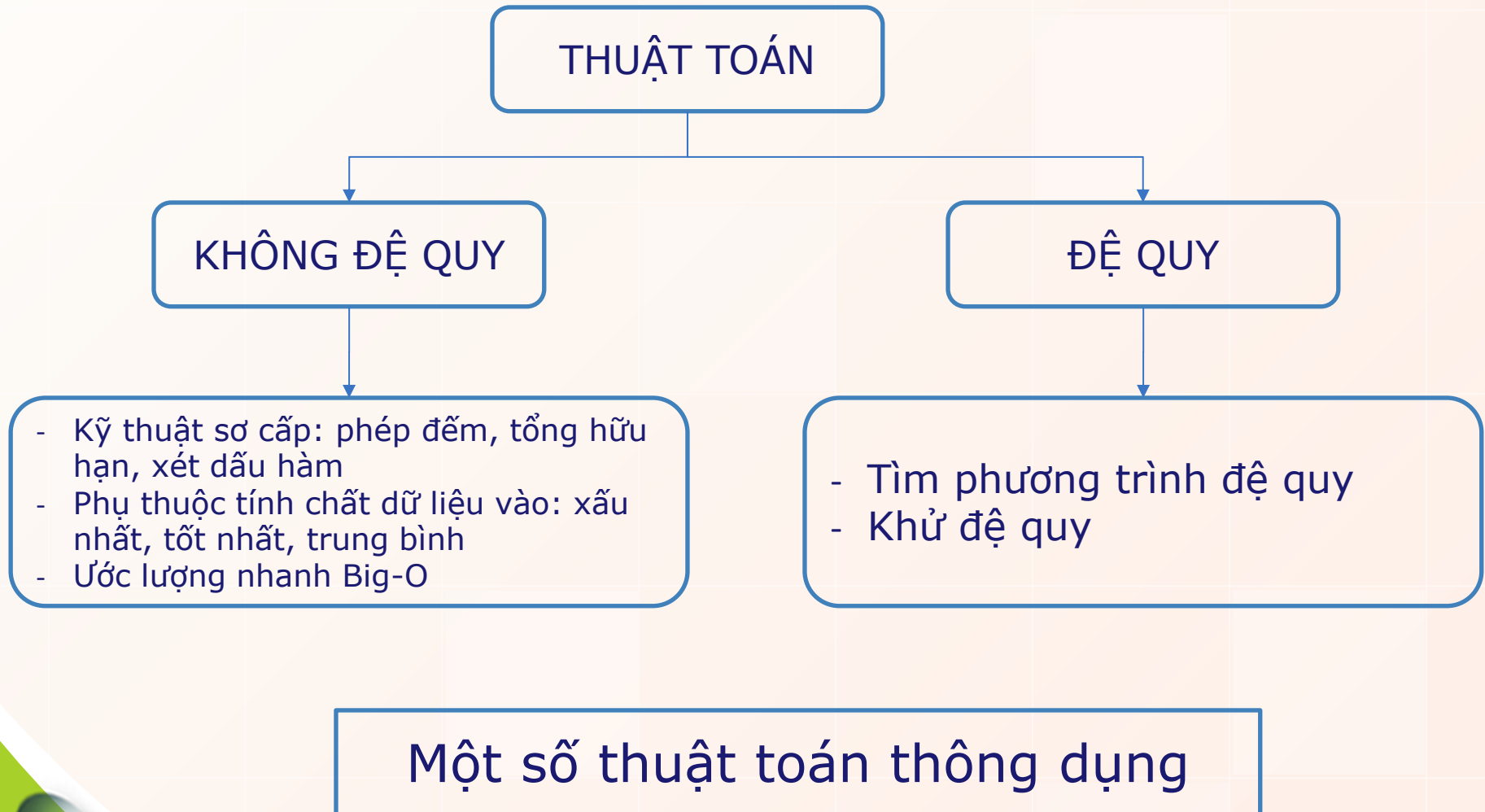
CHƯƠNG 2: PHÂN TÍCH THUẬT TOÁN



L/O/G/O

www.themegallery.com

Đánh giá tính hiệu quả về thời gian



Mathematical Analysis of Non-recursive Algorithms

General Plan for Analyzing the Time Efficiency of Nonrecursive Algorithms

1. Decide on a parameter (or parameters) indicating an input's size.
2. Identify the algorithm's basic operation. (As a rule, it is located in the inner-most loop.)
3. Check whether the number of times the basic operation is executed depends only on the size of an input. If it also depends on some additional property, the worst-case, average-case, and, if necessary, best-case efficiencies have to be investigated separately.
4. Set up a sum expressing the number of times the algorithm's basic operation is executed.⁴
5. Using standard formulas and rules of sum manipulation, either find a closed-form formula for the count or, at the very least, establish its order of growth.

[1]Anany Levitin, Introduction to the Design and Analysis of Algorithms, 3rd Edition, 2014 (trang 62)

Tóm lại

- ❖ Mathematical analysis = Count the number of times “each of operations” or “basic operation” is executed.
- Basic operation = the most important operation, contribute the most to total running time

Running time: Function of the input size

- ❖ Thời gian thực hiện chương trình là một hàm phụ thuộc kích thước dữ liệu vào.
- ❖ Ký hiệu: $T(n)$ với n là kích thước (độ lớn) dữ liệu vào
- ❖ $T(n) \geq 0 \quad \forall n \geq 0$
- ❖ Ví dụ: Chương trình tính tổng của n số có thời gian thực hiện là $T(n) = ?$ (đơn vị: số chỉ thị thực thi/ số phép toán)

Mathematical Analysis of Recursive Algorithms

General Plan for Analyzing the Time Efficiency of Recursive Algorithms

1. Decide on a parameter (or parameters) indicating an input's size.
2. Identify the algorithm's basic operation.
3. Check whether the number of times the basic operation is executed can vary on different inputs of the same size; if it can, the worst-case, average-case, and best-case efficiencies must be investigated separately.
4. Set up a recurrence relation, with an appropriate initial condition, for the number of times the basic operation is executed.
5. Solve the recurrence or, at least, ascertain the order of growth of its solution.

Phân tích thuật toán đệ quy (Analysis of Recursive Algorithms)

❖ Cách 1:

- Thành lập phương trình đệ quy
- Giải phương trình đệ quy
 - Thời gian thực hiện chương trình = nghiệm của phương trình đệ quy

❖ Cách 2:

- Khử đệ quy, phân tích Thuật toán không đệ quy

Thành lập phương trình đệ quy (Recurrence relation)

❖ Dạng tổng quát của phương trình đệ quy

$$T(n) = \begin{cases} C(n) \\ g(T(k)) + f(n) \end{cases}$$

- $C(n)$: thời gian thực hiện chương trình trong trường hợp dừng
- $g(T(k))$ là đa thức của $T(k)$
- $f(n)$ là thời gian phân chia/kết hợp các kết quả

EXAMPLE 1 Compute the factorial function $F(n) = n!$ for an arbitrary nonnegative integer n . Since

$$n! = 1 \cdot \dots \cdot (n - 1) \cdot n = (n - 1)! \cdot n \quad \text{for } n \geq 1$$

and $0! = 1$ by definition, we can compute $F(n) = F(n - 1) \cdot n$ with the following recursive algorithm.

ALGORITHM $F(n)$

//Computes $n!$ recursively

//Input: A nonnegative integer n

//Output: The value of $n!$

if $n = 0$ **return** 1

else return $F(n - 1) * n$

Recurrence equation or
recurrence relation
(or simply a recurrence)

The basic operation of the algorithm is multiplication, whose number of executions we denote $M(n)$

$$\begin{aligned} M(n) &= M(n - 1) + 1 \quad \text{for } n > 0, \\ M(0) &= 0. \end{aligned}$$

Thành lập phương trình đệ quy (Recurrence relation)

- ❖ Ví trùng cứ 1 giờ lại nhân đôi. Vậy sau 5 giờ sẽ có mấy con vi trùng nếu ban đầu có 2 con?

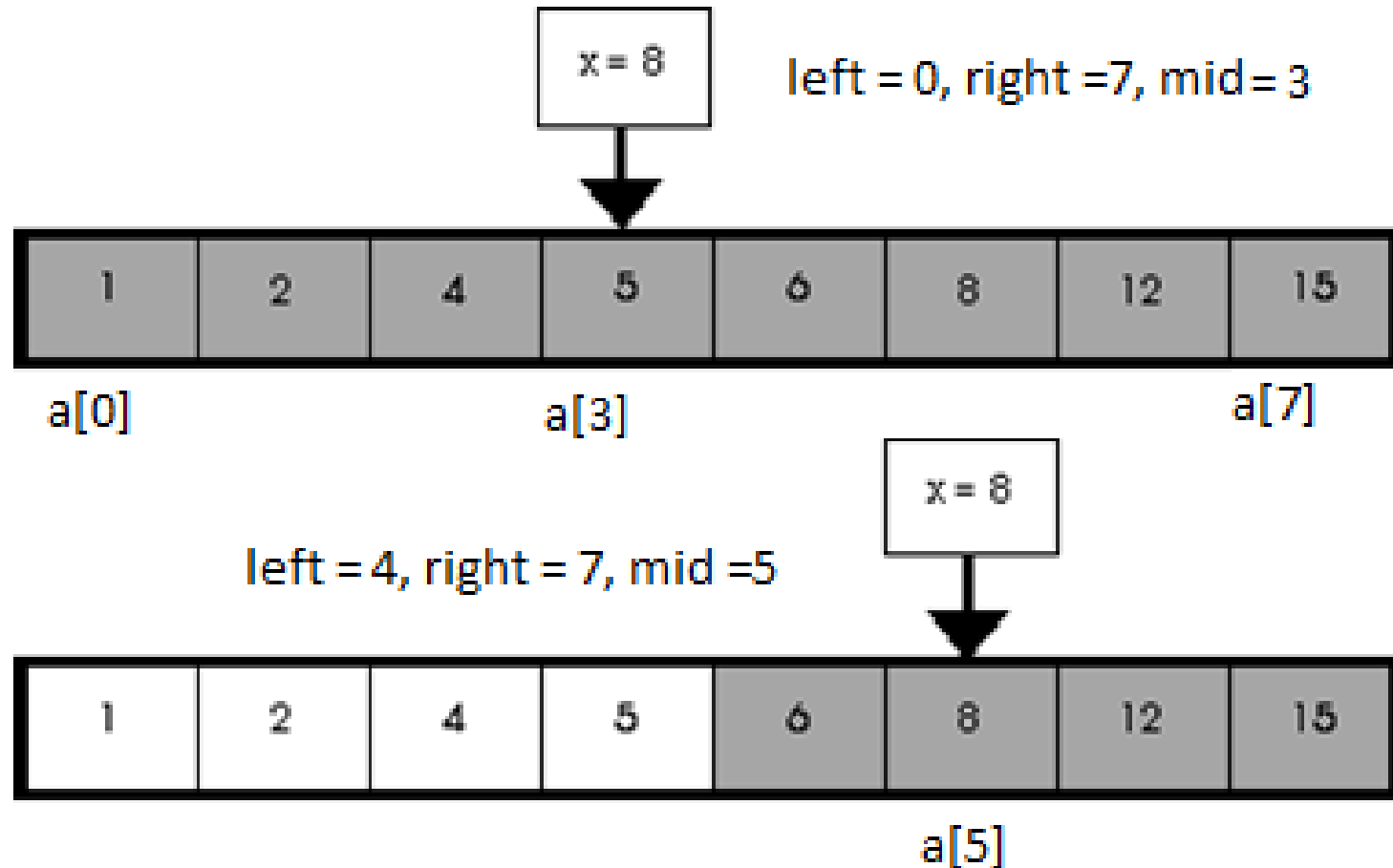
$$T(n) = \begin{cases} C_1 & \text{khi } n = 0 \\ T(n-1) + C_2 & \text{khi } n > 0 \end{cases}$$

- ❖ Ví dụ (dành cho SV tự học/tự nghiên cứu tài liệu)

```
int vitrung(int n)
{
    if (n == 0)
        return 2;
    return 2*vitrung(n-1);
}
```

Hướng dẫn giải tại lớp

✓ Bài 1: Binary Search



Hướng dẫn giải tại lớp

✓ Bài 1: Binary Search

```
int binarySearch(int a[], int n, int l, int r, int x)
{
    if (l > r) return -1;

    int mid = (l+r)/2;

    if (x == a[mid]) return mid;
    if (x < a[mid]) return binarySearch(a, n, l, mid-1, x);
    return binarySearch(a, n, mid+1, r, x);
}
```

✓ Bài 2

Hướng dẫn giải tại lớp

Câu 2 (2 điểm) (G2.) Phân tích thuật toán

Đánh giá độ phức tạp của hàm f được cho bên dưới:

```
int f (int n)
{
    if (n==1)    return 2;
    return 3f(n/2) + 2*log(f(n/2)) - f(n/2) + 1;
}
```

Yêu cầu: Thành lập phương trình đệ quy (kèm giải thích ngắn gọn cách thành lập) và ~~giải phương trình dùng phương pháp truy hồi (còn gọi là thay thế).~~

✓ Bài 3

Hướng dẫn giải tại lớp

2.2. Xét một giải thuật đệ quy như sau:

```
waste (n)  
{  
    if(n==0) return 0;  
    for(i=1 to n)  
        for(j=1 to n)  
            print i,j,n;  
    for(i=1 to 3)  
        waste(n/2);  
}
```

Yêu cầu:

a. Thành lập phương trình đệ quy (hay còn gọi là hệ thức truy hồi) của giải thuật nêu trên, kèm giải thích ngắn gọn cách thành lập.

✓ Bài 4

Câu 3 (2 điểm) (G 2.) Phân tích thuật toán

Cho thuật toán đệ quy để tính tổng $S(n) = 1^3 + 2^3 + \dots + n^3$ như sau:

```
Algorithm S(n)
{  if (n==1) return 1;
    return S(n-1)+n*n*n;
}
```

Yêu cầu:

- Giả sử phép nhân là tác vụ căn bản của thuật toán. Hãy lập phương trình đệ quy biểu diễn chính xác số lần thực hiện tác vụ căn bản trong thuật toán. Giải thích ngắn gọn cách thiết lập phương trình. Lưu ý: Vì yêu cầu là đếm chính xác nên không sử dụng các hằng số không cụ thể như c_1 , c_2 hay ước lượng theo ký hiệu tiệm cận.

Phân tích thuật toán đệ quy (Analysis of Recursive Algorithms)

❖ Cách 1:

- Thành lập phương trình đệ quy
- Giải phương trình đệ quy
 - Thời gian thực hiện chương trình = nghiệm của phương trình đệ quy

❖ Cách 2:

- Khử đệ quy, phân tích Thuật toán không đệ quy

Phân tích thuật toán đệ quy – Giải phương trình đệ quy

- ❖ Truy hồi/Thay thế/Thế ngược
 - ❖ Phương trình đặc trưng
 - ❖ Phương pháp hàm sinh
 - ❖ Định lý Master
 - ❖ Đoán nghiệm và quy nạp
 - ❖ Khác
- ❖ Backward substitution
 - ❖ Characteristic equation
 - ❖ Generating function
 - ❖ Master theorem
 - ❖ Guessing and Induction

Phương pháp Truy hồi/Thay thế/Thế ngược (Backward Substitution)

- ❖ Dùng đệ quy để **thay thế** $T(k)$ ($k < n$) vào phía phải của phương trình
 - ❖ Lặp lại cho đến khi tất cả $T(k)$ được thay thế bởi các biểu thức $T(1)$ hoặc $T(0)$.
 - ❖ $T(1)$ và $T(0)$ luôn là hằng số
- công thức $T(n)$ chứa các số hạng chỉ liên quan tới n và hằng số.

Method of backward substitutions

❖ Ví dụ (dành cho SV tự học/tự nghiên cứu tài liệu)

$$M(n) = M(n - 1) + 1 \quad \text{for } n > 0, \\ M(0) = 0.$$

$$\begin{aligned} M(n) &= M(n - 1) + 1 \\ &= [M(n - 2) + 1] + 1 = M(n - 2) + 2 \\ &= [M(n - 3) + 1] + 2 = M(n - 3) + 3 \end{aligned}$$

substitute $M(n - 1) = M(n - 2) + 1$

substitute $M(n - 2) = M(n - 3) + 1$

$$= \dots = M(n - i) + i = \dots$$

Quá trình kết thúc khi $n - i = 0 \rightarrow i = n$

Khi đó:

$$M(n) = M(n - n) + n = n$$

Method of backward substitutions

❖ Ví dụ (dành cho SV tự học/tự nghiên cứu tài liệu)

$$M(n) = 2M(n - 1) + 1 \quad \text{for } n > 1, \\ M(1) = 1.$$

$$M(n) = 2M(n - 1) + 1 \\ = 2[2M(n - 2) + 1] + 1 = 2^2 M(n - 2) + 2 + 1$$

$$= 2^2[2M(n - 3) + 1] + 2 + 1 = 2^3 M(n - 3) + 2^2 + 2 + 1$$

generally, after i substitutions, we get

$$M(n) = 2^i M(n - i) + 2^{i-1} + 2^{i-2} + \cdots + 2 + 1 = 2^i M(n - i) + 2^i - 1$$

Since the initial condition is specified for $n = 1$, which is achieved for $n - i = 1$ or $i = n - 1$

$$M(n) = 2^{n-1} M(1) + 2^{n-1} - 1 = 2^{n-1} + 2^{n-1} - 1 = 2^n - 1$$

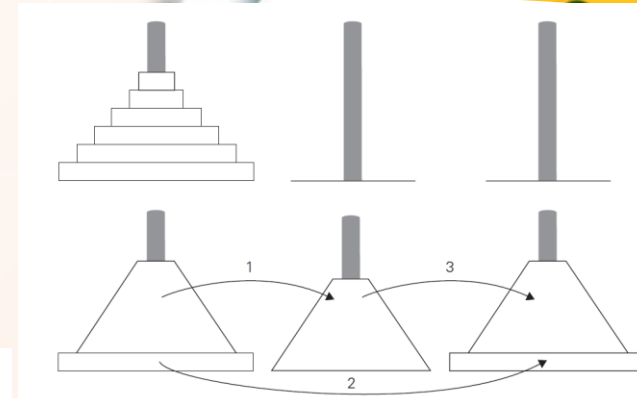


FIGURE 2.4 Recursive solution to the Tower of Hanoi puzzle.

The pattern of the first three sums??

Hướng dẫn giải tại lớp

❖ Bài 1:
$$T(n) = \begin{cases} 0 & \text{khi } n = 1 \\ T(n-1) + 2 & \text{khi } n > 1 \end{cases}$$

❖ Bài 2: Binary Search

$$T(n) = \begin{cases} c_1 & \text{khi } n = 0 \\ T\left(\frac{n}{2}\right) + c_2 & \text{khi } n > 1 \end{cases}$$

❖ Bài 3:

$$T(n) = \begin{cases} c_1 & \text{khi } n = 1 \\ 3T\left(\frac{n}{2}\right) + c_2 & \text{khi } n > 1 \end{cases}$$

Bonus#02.a: 2đ

$$T(n) = \begin{cases} c_1 & \text{khi } n = 1 \\ 3T\left(\frac{n}{2}\right) + c_2 n^2 & \text{khi } n > 1 \end{cases}$$

Phương pháp Đoán nghiệm (The most general method)

1. *Guess* the form of the solution.
2. *Verify* by induction.
3. *Solve* for constants.

Phương pháp Đoán nghiệm (Guessing and Induction)

- ❖ Ta đoán 1 nghiệm $f(n)$ và dùng **chứng minh quy nạp** để chứng tỏ rằng **$T(n) \leq f(n)$, với mọi n**
- ❖ $f(n)$ thường là một trong các hàm quen thuộc như $\log n$, n , n^2 , n^3 , 2^n , $n!$, n^n
- ❖ Đôi khi ta chỉ đoán dạng của $f(n)$ trong đó có vài tham số chưa xác định và trong quá trình quy nạp, ta sẽ tìm ra giá trị thích hợp cho các tham số.

❖ Ví dụ *(dành cho SV tự học/tự nghiên cứu tài liệu)*

$$T(n) = \begin{cases} C_1 & \text{khi } n = 0 \\ T(n-1) + C_2 & \text{khi } n > 0 \end{cases}$$

Ta đoán $f(n) = a \cdot n + b$

- B1: Với $n = 0$, $T(0) = C_1$ và $f(0) = b$,

Để có $T(0) \leq f(0)$ thì Chọn $C_1 \leq b$

- B2: Giả sử $T(k) \leq f(k)$, $\forall k < n$.

- B3: Ta cm $T(n) \leq f(n)$, tại n

$$T(n) = \begin{cases} C_1 & \text{khi } n = 0 \\ T(n-1) + C_2 & \text{khi } n > 0 \end{cases}$$

Ta đoán $f(n) = a \cdot n + b$

- B3: Cần cm $T(n) \leq f(n)$,
 - Áp dụng giả thiết quy nạp với $k = n-1 < n$ ($n > 0$)
 - ta có $T(n-1) \leq f(n-1) = a \cdot (n-1) + b$

$$T(n) = T(n-1) + C_2 \leq a(n-1) + b + C_2$$

$$T(n) \leq an + b + C_2 - a$$

$$T(n) = \begin{cases} C_1 & \text{khi } n = 0 \\ T(n-1) + C_2 & \text{khi } n > 0 \end{cases}$$

Ta đoán $f(n) = a \cdot n + b$

B3: Cần cm $T(n) \leq f(n), \forall n$

$$T(n) \leq an + b + C_2 - a$$

Nếu: $C_2 - a \leq 0$

Thì: $T(n) \leq an + b = f(n)$

$$T(n) = \begin{cases} C_1 & \text{khi } n = 0 \\ T(n-1) + C_2 & \text{khi } n > 0 \end{cases}$$

- Để tìm a, b, ta giải hệ:
$$\begin{cases} b \geq C_1 \\ a \geq C_2 \end{cases}$$
- Suy ra: $b = C_1, a = C_2$
- Ta có: $T(n) \leq C_1 + C_2 n \quad \forall n$
- $T(n) = O(n)$

Hướng dẫn giải tại lớp (Q22, Q25)

❖ Bài 1:

$$T(n) = \begin{cases} c_1 & \text{khi } n = 1 \\ 3T\left(\frac{n}{2}\right) + c_2 n^2 & \text{khi } n > 1 \end{cases}$$

Bonus#02.b: 1.5đ

❖ Bài 2:

$$T(n) = \begin{cases} c_1 & \text{khi } n = 1 \\ 2T\left(\frac{n}{2}\right) + c_2 n & \text{khi } n > 1 \end{cases}$$